

请输入文本设计报告

哈尔滨工业大学（深圳） 请输入文本队

李建霖、王娅藟、黄阅、贾云博

1 设计简介

我们的 CPU 使用 Verilog 实现了基于 LoongArch32R (LA32R) 指令集的乱序双发射处理器（我们称其为“意味核”）。意味核前端用块（最多四条指令）做取指单位，并用 FTQ/IB 把预测、取指、后端拆开，配备 uBTB、fallback BTB、FTB、TAGE 与 RAS 组成的多级分支预测；后端以 ROB 编号作为重命名标签，将保留站按指令类型拆成三类（ALU/访存/乘除杂项），支持每拍最多两条指令提交。存储包含 16KB Icache、16KB Dcache 与 128KB L2 cache，以 cache 一行为单位经 AXI 与 SoC 外设通信。功能上覆盖 LA32R 用户态/特权态主要指令（含 `ibar/dbar/preld`），性能测试上板实现频率为 **75 MHz**（WNS +0.052 ns）。

2 设计方案

2.1 总体设计思路

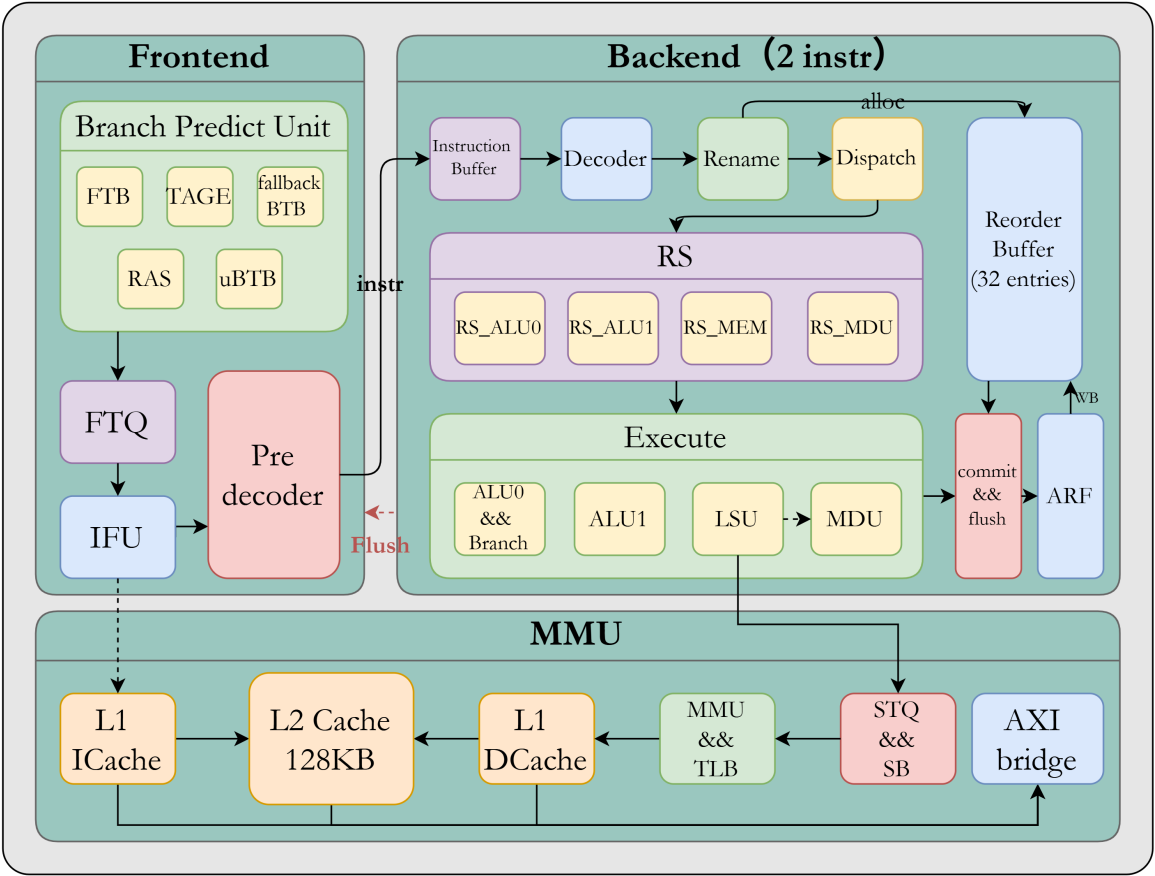


图 1 CPU 设计架构图

前端可划分为分支预测与取指两大部分：分支预测包含 uBTB、fallback BTB、FTB、TAGE、RAS 等组件；取指路径为 FTQ → IFU（含预译码）→ ICache → 指令缓冲（IB），每拍最多向 IB 填入 4 条指令。

后端从流水级上看分为译码、重命名、分发、保留站发射、执行与提交；从功能上看包含译码器（decoder×2）、指令缓冲（IB）、寄存器别名表（RAT）、架构寄存器堆（ARF）、重排序缓存（ROB）、分发单元（dispatch）、三类保留站（RS_ALU×2、RS_MEM、RS_MDU）以及执行单元 ALU×2、LSU、MDU，并由提交级与 ctrl（控制冲刷）统一处理误预测、异常与特权副作用。

指令执行过程大致如下：

1. **译码：**每拍从 IB 最多取两条指令，生成 FU 类型、操作码、立即数、特权向量与例外信息；
2. **重命名：**查询/更新 RAT，以 ROB 编号作为源/目的寄存器的重命名标签（无 PRF 与 freelist），并成对分配 ROB 项（双发所以成对分配）；

3. 分发：从 ROB 补齐就绪操作数，将指令路由至对应保留站；
4. 发射与执行：ALU 保留站乱序发射到双 ALU（含分支判定）；访存与乘除杂项经各自 FIFO 保留站进入 LSU/MDU；结果写回 ROB；
5. 提交：检查 ROB 队头至多两条指令是否可退休，完成写 ARF、CSR/TLB 维护、Store Buffer 入队、分支训练以及异常/误预测冲刷。

各容量概览见表 1。

表 1 容量概览

结构	规格
取指 / 后端宽度	4 宽基本块取指 / 2 宽译码与提交
ROB	32 项（分奇偶两部分，一部分 16 项，共 16 对）
IB / FTQ	16 项 / 16 项
RS	ALU×2 各 4 项；MEM 4 项；MDU 2 项
STQ / Store Buffer	16 / 8
Icache / Dcache	各 16 KB，4 路 ×128 组 ×32 B，VIPT
L2	128 KB，2 路 ×2048 组 ×32 B
TLB	主 TLB 32 项双查口；I/D L1 微 TLB 各 8 项
BPU	uBTB 16；fallback BTB 64；FTB 4 × 2048；TAGE 8K+4×1K；RAS 32 双栈

2.1.1 指令实现

按照指令类型，意味核实现了如下指令（见表 2）。其中 `ibar/dbar` 译码为特权屏障标记，提交时等待 Store Buffer 排空后发起重取冲刷；`preld` 被识别为已知指令并以空操作退休（不做硬件预取），从而不触发 INE。

表 2 指令实现

指令类型	指令名称
算术运算类指令	<code>add.w, sub.w, addi.w, slt, sltu, slti, sltui, lu12i.w, pcaddu12i, and, or, nor, xor, andi, ori, xori, mul.w, mulh.w, mulh.wu, div.w, div.wu, mod.w, mod.wu</code>
移位运算类指令	<code>sll.w, srl.w, sra.w, slli.w, srli.w, srli.w</code>
分支跳转指令	<code>beq, bne, blt, bltu, bge, bgeu, b, bl, jirl</code>
普通访存指令	<code>ld.b, ld.bu, ld.h, ld.hu, ld.w, st.b, st.h, st.w</code>
原子访存指令	<code>ll.w, sc.w</code>
屏障与预取	<code>ibar, dbar, preld</code>
特权指令	<code>csrrd, csrwr, csrchg, tlbsrch, tlbrd, tlbwr, tlbfll, invtlb, cacop, ertn, idle</code>
系统异常指令	<code>syscall, break</code>
计时器相关指令	<code>rdcntid, rdcntvl.w, rdcntvh.w</code>
读取配置指令	<code>cpucfg</code>

2.1.2 CSR 实现

意味核实现了以上全部指令所需的 CSR，具体如表 3。未实现编号的 CSR 读返回 0、写忽略。部分低风险 CSR（如 TICLR、SAVE0-3、TID）采用 L0 写路径，提交后无需全局重取。

表 3 CSR 实现

地址	含义	名称	地址	含义	名称
0x0	当前模式信息	CRMD	0x30-33	数据保存	SAVE0-3
0x1	例外前模式信息	PRMD	0x40	定时器编号	TID
0x2	扩展组件使能	EUEN	0x41	定时器配置	TCFG
0x4	例外配置	ECFG	0x42	定时器值	TVAL
0x5	例外状态	ESTAT	0x44	定时中断清除	TICLR
0x6	例外返回地址	ERA	0x60	LLBit 控制	LLBCTL
0x7	出错虚地址	BADV	0x88	TLB 重填入口	TLBREENTRY
0xC	例外入口地址	EENTRY	0x180-181	直接映射窗口	DMW0-1
0x10	TLB 索引	TLBIDX	0x18	ASID	ASID
0x11	TLB 表项高位	TLBEHI	0x19	低半目录基址	PGDL
0x12	TLB 表项低位 0	TLBELO0	0x1A	高半目录基址	PGDH
0x13	TLB 表项低位 1	TLBELO1	0x1B	全局目录基址	PGD
0x20	处理器编号	CPUID			

2.1.3 例外与中断

本处理器支持 LoongArch-32R 手册要求的主要例外与中断，如表 4。例外在流水中附着于指令，于提交级按优先级编码后写入 CSR 并冲刷前端。而中断则附着在即将提交的指令上处理。idle 提交后冻结取指，直至中断到来。

表 4 例外与中断

Ecode	Esubcode	例外代号	例外类型
0x0	0	INT	中断
0x1	0	PIL	load 操作页无效例外
0x2	0	PIS	store 操作页无效例外
0x3	0	PIF	取指操作页无效例外
0x4	0	PME	页修改例外
0x7	0	PPI	页特权等级不合规例外
0x8	0	ADEF	取指地址错例外
0x9	0	ALE	地址非对齐例外
0xB	0	SYS	系统调用例外
0xC	0	BRK	断点例外
0xD	0	INE	指令不存在例外
0x3F	0	TLBR	TLB 重填例外

2.1.4 地址翻译模式

意味核支持直接地址翻译、直接映射地址翻译与页表映射地址翻译三种模式。

当 CSR.CRMD 中 DA=1 且 PG=0 时为直接地址翻译，物理地址等于虚拟地址。当 DA=0 且 PG=1 时进入映射模式：若虚地址命中有效 DMW 窗口，则按窗口配置拼接物理地址高位；否则查询 TLB。命中有效 TLB 表项则完成翻译，否则触发 TLB 重填例外。取指与访存通路各自经过 MMU 门面，并配备 L1 微 TLB 加速常用 4KB 页翻译。

2.2 前端设计

前端采用取基本块结构，主要包括分支预测器（BPU）、取指目标队列（FTQ）、取指单元（IFU，含预译码）与一级指令缓存（ICache）。BPU 每拍最多对一个起始地址产生一个预测块，块内最多 4 条指令；出于地址翻译正确性考虑，保证块内指令不跨 4KB 页。

2.2.1 分支预测器

BPU 实现条件分支、函数调用与无条件跳转的目标预测、取指宽度截断以及条件分支方向预测。核心组件包括：

- **uBTB**: 16 项全相联，当拍返回，提供无空泡基线预测块（默认取指宽度 4）；
- **fallback BTB**: 64 项直接映射，在 uBTB 未命中时提供 P0 后备目标，避免 CAM（Content-Addressable Memory）拉长关键路径；
- **FTB**: 四路组相连、每路 2048 组，共 8192 项，使用 BRAM 实现，保存基本块分支元数据；

- **TAGE**: 基础表 8192 项 (2 bit 饱和计数) 与 4 张标记表 (各 1024 项, 12 bit tag), 全局历史长度分别为 11/23/53/112;
- **RAS**: 32 项, 前端推测栈与后端提交栈双栈结构, 分别服务预测期压弹与提交期校正。

BPU 流水 (P0/P1):

- **P0 (当拍)**: 以当前 PC 查询 uBTB / fallback BTB, 生成默认预测块并送入 FTQ, 保证取指不因主预测延迟而空泡;
- **P1 (次拍)**: FTB 与 TAGE 返回。若主预测与 P0 块不一致, 则发出前端重定向并修正 FTQ 中上一拍条目。

FTB 条目主要包括: 由预测块起始地址决定的 tag、顺序下落地址 (fall-through)、跳转目标、分支类型等。FTB/TAGE 的训练以提交级分支结果为准, 经训练队列回写, 保证推测路径不会污染架构正确的预测表。当基本块出现无条件跳转、条件分支首次 taken、或分支信息错误等情况时, 更新或分配 FTB 条目。

2.2.2 取指目标队列 FTQ

FTQ 深度为 16, 采用三指针环形结构 (BPU 写指针、IFU 读指针、提交释放指针), 在分支预测与取指之间解耦, 并缓存 BPU 训练元数据。BPU 在 P0 即将预测块写入 FTQ; 若 P1 主预测纠正, 则就地修改对应条目。提交阶段按程序序释放 FTQ 项, 并把分支结果送回 BPU 训练。每个块最多 4 条指令, 与后端 ROB 深度相匹配, 避免前端过度推测导致资源浪费。

2.2.3 取指令单元与取指流水

IFU 从 FTQ 读取预测块, 经 MMU 做取指地址翻译; 无异常时向 ICache 发起整行取指请求。取指流水可概括为:

- **PRE**: 接收 FTQ 块, 发起虚地址翻译, 并在允许时向 ICache 发请求; 支持 FTQ→Icache 同拍直发 (在无需主 TLB 的直接翻译场景下);
- **IF**: 等待 ICache 返回 cache line, 按预测块起止切割指令, 写入 IB;
- **预译码重定向**: 对不依赖寄存器的强制跳转做前端检查, 若发现漏预测则冲刷前端取指/预测流水并重定向;
- **冲刷协同**: 后端冲刷时, 根据是否存在在途 Icache 读事务决定是否丢弃迟到返回, 并清空流水寄存器。

ICache 失效 (如 **cacop**) 时, IFU 同步清理 PRE/IF 与行缓冲, 避免自修改代码路径读到陈旧指令。

2.3 指令缓冲（IB）模块设计

指令缓冲用于实现前后端解耦，使前端取指与后端执行互不阻塞。IB 深度为 16，每拍最多推入 4 条指令、弹出 2 条指令供译码使用。每项包含：指令有效位、PC、指令码、分支预测信息以及取指阶段附着的例外信息等。

由于入队宽度可变，IB 将空闲项不足 4 时视为满，反压前端取指，而队列为空时向后端给出无效气泡，保证流水握手完整。此外，IB 写入口与译码/重命名 ready 解耦，避免冷启动时因后端尚未就绪而损失本可同拍消费的取指带宽。

2.4 后端设计

后端为 2 宽乱序执行流水：译码 → 重命名 → 分发 → 保留站发射 → 执行写回 → 提交。与统一 Issue Queue 的顺序双发射不同，本设计使用异构保留站分离 ALU/MEM/MDU 发射策略，并以 ROB 编号作为重命名标签，结果驻留 ROB，无独立物理寄存器堆。

2.4.1 译码模块设计（ID）

每拍从 IB 取出至多两条指令，例化两个组合译码器。译码产生 FU 类型（ALU/MEM/MDU）、独热操作码、立即数、源与目的寄存器信息，以及 CSR、TLB、cacop、ibar 等特权向量；同时检测 INE、SYS、BRK、IPE 等译码例外。分支指令在译码阶段标记为 ALU 类，实际方向与目标在 ALU 中计算，最终于提交级与预测结果比对。

2.4.2 重命名映射表（RAT）与重命名级（RA）

RAT 为 32 项结构，记录各逻辑寄存器是否 busy 及其映射的 ROB 编号。重命名级主要完成：

1. 向 ROB 申请成对编号（奇偶双体，保证双发射整对整对分配）；
2. 读 RAT/ARF 获得源操作数或生产者 ROB 索引；同拍双指令之间的 WAR/WAW/RAW 通过旁路修正；
3. 若指令写寄存器，则更新 RAT 的 busy 与 ROB 映射；
4. 将静态信息与重命名结果锁存，送往分发级。

提交时，若退休指令仍是某逻辑寄存器的最新生产者，则清除对应 busy，完成释放。误预测或异常冲刷时，RAT 随全局冲刷恢复到最后正确提交的那一拍的状态。

2.4.3 分发与保留站发射（Dispatch / RS）

分发单元从 ROB 读取尚未就绪的操作数，并将指令路由到三类保留站：

- RS_ALU0 / RS_ALU1：各 4 项，乱序挑选就绪项，可同拍向两路 ALU 双发；

- **RS_MEM**: 4 项, FIFO 为主, 并允许有限的 load 越过未就绪 load, 每拍最多入队 1 条访存;
- **RS_MDU**: 2 项, 严格 FIFO, 服务乘除、CSR 读、rdcnt、cpucfg、invtlb 打包等。

源操作数就绪且目标 FU 可接收时发射。CSR 写、TLB 维护、异常等副作用不在发射级提交, 而由提交级统一处理, 实现精确异常。

2.4.4 执行模块设计 (EXE)

- **ALU0/ALU1**: 单周期算术逻辑与分支判定, 结果写回 ROB, 并给出“是否跳转”标志位与跳转目标供提交比对;
- **LSU**: AGU + DCache 两级。AGU 计算虚地址并经 MMU 翻译、检查对齐/页例外; DC 级完成 Store Buffer 前递、Dcache 访问与 miss 处理。支持命中快速写回与 store→load 安全前递;
- **MDU**: 非流水多周期单元。乘法为 DSP 流水 (约 3 拍), 除法为 CLZ 快速除法 (约 5-10 拍), CSR 读类指令在此取数, TLB 相关指令先在执行级占位, 然后提交级真正维护。

四路 FU 写回 ROB 对应项并置 complete。访存推测 store 先进入 STQ (深度 16), 按 ROB 序在提交后转入 Store Buffer。

2.4.5 重排序缓存 (ROB)

ROB 共 32 项, 奇偶双体环形队列, 是重命名标签、结果缓存与提交序的核心。各项保存 PC、执行结果、访存/特权信息、预测元数据与异常向量等。dispatch 侧提供多读口补操作数, FU 侧多写口完成, commit 侧读取队头。分配时清 complete, 写回时置位。冲刷时按提交点回收尾指针之后的投机项。

2.4.6 提交模块设计 (COMMIT) 与冲刷

提交级每拍最多退休两条指令, 并在下列情况退化为单提交或停提交: 后条为需串行的特权/ll/sc/屏障、前条异常或 ertn、仅一条就绪等。退休时写 ARF、按需写 CSR/TLB、推入 Store Buffer、更新 LLBit, 并向 BPU/FTQ 回送训练。

误预测、例外、ertn 以及 CSR/TLB/cacop/ibar 等导致的重取, 均由提交级生成冲刷类型, 经 ctrl 广播到前端与后端, 实现单一恢复点。这与在执行级立即重定向的设计不同, 换取了精确异常与特权顺序的简洁性, 并以更高的双提交率与预测精度弥补部分延迟。

2.5 转接桥模块 (AXI Line Bridge)

axi_line_bridge 位于 L2 与 SoC AXI4 主口之间, 负责 32B cache 行与 32 bit AXI 突发之间的协议转换, 并为 I/D 两侧提供独立读端口。

主要行为如下：

- **行读：**ARLEN=7（8 拍 ×32 bit INCR）。桥接每凑满 128 bit 向上游回一拍，一行共 2 拍数据，最后一拍拉高 `ret_last`；
- **行写：**上游以 2 拍 128 bit 送入整行，桥接缓冲后以 AWLEN=7 写出；支持 AW/W 通道并行握手；
- **Uncached 单拍：**按真实访问宽度设置 `arsize/awsize`；设备写必须等待 B 响应完成，保证外设寄存器访问次序；
- **双 outstanding 读：**取指读与数据读使用不同 `arid` (0/1)，R 通道按 `rid` 分流；
- **写后读序：**任一写尚未收到 B 前，不发起新的 AR，避免 victim 写回后同地址 refill 读到旧数据；
- **行写 B 解耦：**行写数据发完即可继续接收，降低写回气泡。

顶层 `core_top` 对外呈现标准 AXI4 接口。

2.6 Cache 与访存层次设计

意味核采用 Icache/Dcache + 统一 L2 + AXI 桥的层次结构，并在核内用 Store Queue 与 Store Buffer 分离推测写（STQ 猜它会写，就是投机的意思）与提交写（SB 确实要写）。

2.6.1 一级指令缓存（ICache）

ICache 容量 16KB，组织为 4 路 ×128 组 ×32B 行，采用 VIPT 查询，面向整行取指。命中时向 IFU 返回一行指令数据；缺失时向 L2 请求填行。支持 `cacop` 失效与 `miss` 途中的 `refill_abort`：当同组维护请求到来时，丢弃迟到的安装，避免自修改场景下装入陈旧行。替换在无效路优先策略下进行，存储使用 BRAM。

2.6.2 一级数据缓存（DCache）

DCache 与 ICache 相似（16KB，4 路 ×128×32B，VIPT），面向 LSU 的 load/store 访问。具备非阻塞 `miss` 能力，MSHR 深度为 2；命中路径支持同行 store 合并，降低写后写/写后读气泡。缺失时先查 L2；若需替换脏行，则按地址型维护把写回推进到 L2。`cacop` 在完成 L1 写回/失效后，继续请求 L2 对同一物理行做写回并失效，以满足 Linux DMA 与缓存几何一致性。

2.6.3 二级统一缓存（L2）

L2 为 128KB 的 Icache 与 Dcache 共用的缓存，2 路 ×2048 组 ×32B，写回法。Icache 与 Dcache 有 `miss` 时都会走 L2：Dcache 要 `refill` 时，支持数据边到边往 Dcache 送，load `miss` 能更快结束；I-miss 发现同一行上有 UC 写或 cache 维护写在进行时，不允许这次 I-miss 把那份旧行写回。L2 `miss` 则经 `axi_line_bridge` 访问内存与外设。

2.6.4 STQ 与 Store Buffer

- **STQ (深度 16)**: 保存尚未提交的 store, 维护顺序, 提供 load 前递查询; 指令提交后按 ROB 编号释放;
- **Store Buffer (深度 8)**: 提交后写缓冲, 可做行聚合与同字旁路合并, 再写 Dcache/L2; 冲刷时不清空, 保证已提交的访存操作不丢失。

LSU 优先检查 SB/STQ 前递, 再访问 Dcache; 命中可走旁路提前完成 ROB, 提升访存密集的双提交率。

2.7 地址翻译设计

2.7.1 MMU 模块

mmu 为取指与访存提供双通道翻译: 根据 CSR 中的 CRMD/DMW/ASID 等配置, 在直接映射、DMW 窗口与 TLB 页表映射之间选择路径, 并向 IFU/LSU 返回物理地址、MAT (cached/uncached) 属性与例外信息。

2.7.2 TLB 与微 TLB

主 TLB 为 32 项全相联结构, 提供指令地址与数据地址两个查询口 (当拍返回), 并支持 tlbsrcch/tlbbrd/tlbwr/tlbfill/invtlb 维护端口。在主 TLB 前插入 I/D 各 8 项的 L1 微 TLB, 加速常用 4KB 页翻译; 大页等情形透传主 TLB。

TLB 相关例外 (TLBR、PIL/PIS/PIF、PPI、PME 等) 在翻译通路产生后附着于对应取指/访存指令, 最终在提交级写入 BADV/ESTAT 等 CSR, 并跳转 EENTRY 或 TLBREENTRY。

3 设计结果

3.1 设计交付物说明

- score.xlsx	Excel: 功能/性能测试得分
- design.pdf	PDF: 设计文档
- .gitlab-ci.yml	CI/CD 配置
-- src/	
-- mycpu/	我们的 CPU 完整源码
-- xilinx_ip/	我们的 CPU 没有使用 IP 核
-- vivado_cannot/	无
-- perf_clock.json	性能测试 CPU 主频, 75MHZ
-- bit/	各项测试生成的 bit 文件
-- show/	决赛展示内容 (暂时为空)

3.2 设计演示结果

以下是我们的初赛成绩：

表 5 初赛功能与性能分（CI 的结果）

项目	结果	说明
功能分	100.000000	58/58 点（最差 seed）；3/3 seed 通过
IPC 比值	1.934633120	-
系统计数器比值	4.438258572	-
性能测试用例	20/20 PASS	-
FPGA 性能实现频率	75.001875 MHz	cpu_clk WNS +0.052 ns

4 参考设计说明

1. 前端：满洋学长 / 404 NOT FOUND 队

我们重点参考了满洋学长的前端与分支预测相关设计（含 TAGE 等思路）我们结合本核负载对表项容量、历史长度与训练时机进行了调整，将分支预测率提高到 98%，这对多级流水的 IPC 有极大好处。

源代码 URL: <https://github.com/HITSZ-NSCSCC22/mycpu>

2. * 重点思路参考：Cache-HIT-100 队（朱炫翰、蔡源航、房效民、林亮宇）

从前端来说，主要参考其解耦前端结构（BPU / FTB / FTQ / IFU / 预译码等）以及相关报告中的取指与预测流水划分，并完成了学长未完成的 ubtb。

从后端来说，Cache-HIT-100 团队已经具备了乱序的能力。只差唤醒。所以我们着重借鉴了 ROB、RS 等一系列设计，实现了本乱序核。

源代码 URL: <https://gitee.com/zxhzxhPhD/cache-hit-100-so-c>

3. 后端：马家沟河队 Mariver (cpucore-mariver)

参考其乱序后端组织思路，包括重命名、分发、保留站/发射、ROB 写回与提交等阶段划分及执行通路设计。因为其为 mips 架构，所以我们迁移适配到 LA32R 与本核接口。源代码

URL: <https://github.com/HIT-MaRiver-mips/cpucore-mariver>

设计报告: <https://github.com/HIT-MaRiver-mips/report-mariver>

4. openLA500

主要参考流水线异常处理、地址翻译与 uncached 访问相关思路；乘除单元与 AXI 访问次序方面亦有借鉴。其中处理定时器中断的思路被我们用于解决功能测试第 49 个测试点。

源代码 URL: <https://gitee.com/loongson-edu/open-la500>

5 参考文献

1. 汪文祥, 邢金璋. CPU 设计实战 [M]. 北京: 机械工业出版社, 2021.

2. 姚永斌. 超标量处理器设计 [M]. 北京: 清华大学出版社, 2014.